Question 7 - Continuous deployment

Question 7: Configure continuous deployment using Kubernetes by integrating the deployment with the CI/CD pipeline and automating manifest application.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64) with Jenkins 2.580.2 LTS (systemd service, OpenJDK 21), Docker Engine 29.9.3 and Minikube (Kubernetes v1.37.0). Source, logs and screenshots: https://github.com/ksamirdev/ensp461-lab2.

Pipeline design

Stage	What it does
Checkout	Jenkins polls GitHub every 2 minutes (`H/2 * * * *`) and checks out `main`
Test	creates a venv and runs the pytest suite; a failing test stops the pipeline
Build image	`docker build --build-arg APP_VERSION=2.1.<build>`
Push image	pushes `localhost:5001/ensp461-lab2:2.1.<build>` to the lab registry
Deploy to Kubernetes	applies ConfigMap and Service, applies the Deployment with the build's image, records a change cause, waits for the rollout
Smoke test	calls `/health` through the NodePort until it gets the new version 5 times in a row (30 s limit)
Post	on failure, `kubectl rollout undo` restores the previous revision

Jenkinsfile

```
pipeline {
    agent any
    options { timestamps(); disableConcurrentBuilds() }
    triggers { pollSCM('H/2 * * * *') }
    environment {
        PUSH_REPO = 'localhost:5001/ensp461-lab2'      // registry as seen from the Docker host
        PULL_REPO = 'lab2-registry:5000/ensp461-lab2'  // same registry as seen from the cluster node
        VERSION = "2.1.${BUILD_NUMBER}"
    }
    stages {
        stage('Test') {
            steps {
                sh '''
                    python3 -m venv .venv
                    .venv/bin/pip install -q -r app/requirements.txt pytest
                    .venv/bin/python -m pytest -q tests
                '''
            }
        }
        stage('Build image') {
            steps { sh 'docker build --build-arg APP_VERSION=$VERSION -f docker/Dockerfile -t
$PUSH_REPO:$VERSION .' }
        }
        stage('Push image') {
            steps { sh 'docker push $PUSH_REPO:$VERSION' }
        }
        stage('Deploy to Kubernetes') {
            steps {
                sh '''
                    kubectl apply -f k8s/configmap.yaml -f k8s/service.yaml
                    sed "s#image: .*#image: $PULL_REPO:$VERSION#" k8s/deployment.yaml | kubectl apply -f -
                    kubectl annotate deployment/lab2-web kubernetes.io/change-cause="jenkins build
$BUILD_NUMBER: $VERSION" --overwrite
                    kubectl rollout status deployment/lab2-web --timeout=180s
                '''
            }
        }
        stage('Smoke test') {
            steps {
                // Old pods keep serving during their preStop drain after the rollout
                // reports success, so require several consecutive new-version answers.
                sh '''
                    NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[0].address}')
                    ok=0
                    for i in $(seq 30); do
                        BODY=$(curl -fsS http://$NODE_IP:30080/health || true)
                        echo "$BODY"
                        if echo "$BODY" | grep -q "\\"version\\":\\"$VERSION\\""; then ok=$((ok+1)); else
ok=0; fi
                        [ $ok -ge 5 ] && exit 0
                        sleep 1
                    done
                    echo "version $VERSION not served consistently within 30s"
                    exit 1
                '''
            }
        }
    }
    post {
        success { echo "Deployed ensp461-lab2:${VERSION} to Minikube" }
        failure {
            echo 'Deployment failed, rolling back to the previous revision'
            sh 'kubectl rollout undo deployment/lab2-web || true'
        }
    }
}
```

Jenkins wiring

`scripts/q7-setup.sh` connects the Jenkins service from Q0 to the rest of the lab:

- adds the `jenkins` user to the `docker` group, so builds can build and push images;
- writes a flattened kubeconfig for the Minikube cluster to `/var/lib/jenkins/.kube/config` (verified with `sudo -u jenkins kubectl get nodes`);
- disables the setup wizard and installs `workflow-aggregator`, `git`, `pipeline-stage-view` and `timestamper` with the plugin installation manager;
- installs two init scripts from `jenkins/`: `security.groovy` (admin account `samir`, anonymous read for screenshots on this local machine) and `job.groovy` (pipeline job `ensp461-lab2-cd` from the repository's `Jenkinsfile`).

The job is created as code, so the whole CD setup can be rebuilt from the repository.

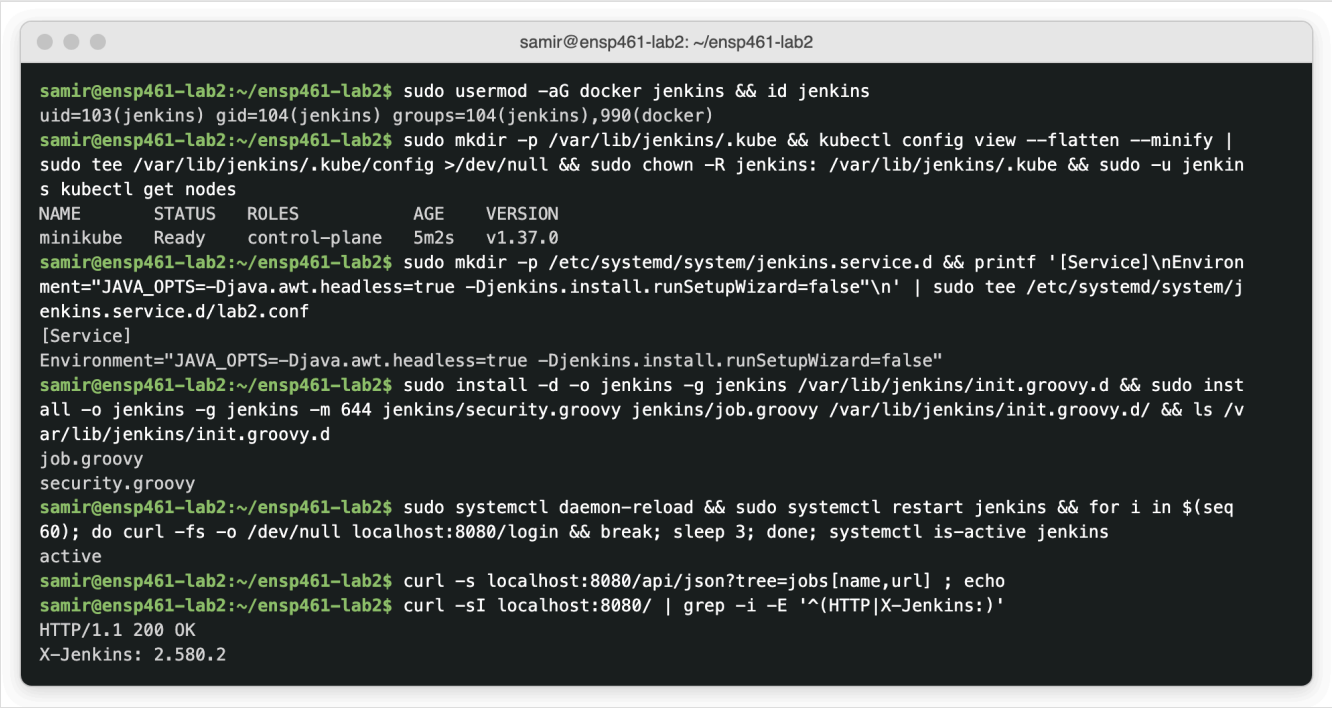


Fig: Jenkins wiring: docker group, kubeconfig, init scripts, restart

Builds

Build	Trigger	Commit	Version	Result
#1	SCM change	`Make Jenkins security init idempotent`		ABORTED during checkout
#2	SCM change	`Add Q7 script that records a finished Jenkins build` (4 commits)	2.1.2	SUCCESS, 176 s
#3	SCM change	`Report process uptime in /health`	2.1.3	SUCCESS, 27 s
#4	SCM change	`Deploy by applying the manifest with the new image`	2.1.4	FAILURE at smoke test, rolled back automatically
#5	SCM change	`Wait for the new version to be served consistently in smoke test`	2.1.5	SUCCESS, 30 s

Build #1 was cut off because the setup script restarted Jenkins while it was cloning; it was aborted and the next push started build #2. Build #2 took 176 s because it downloaded the Python base image and packages; with the cache warm, build #3 took 27 s.

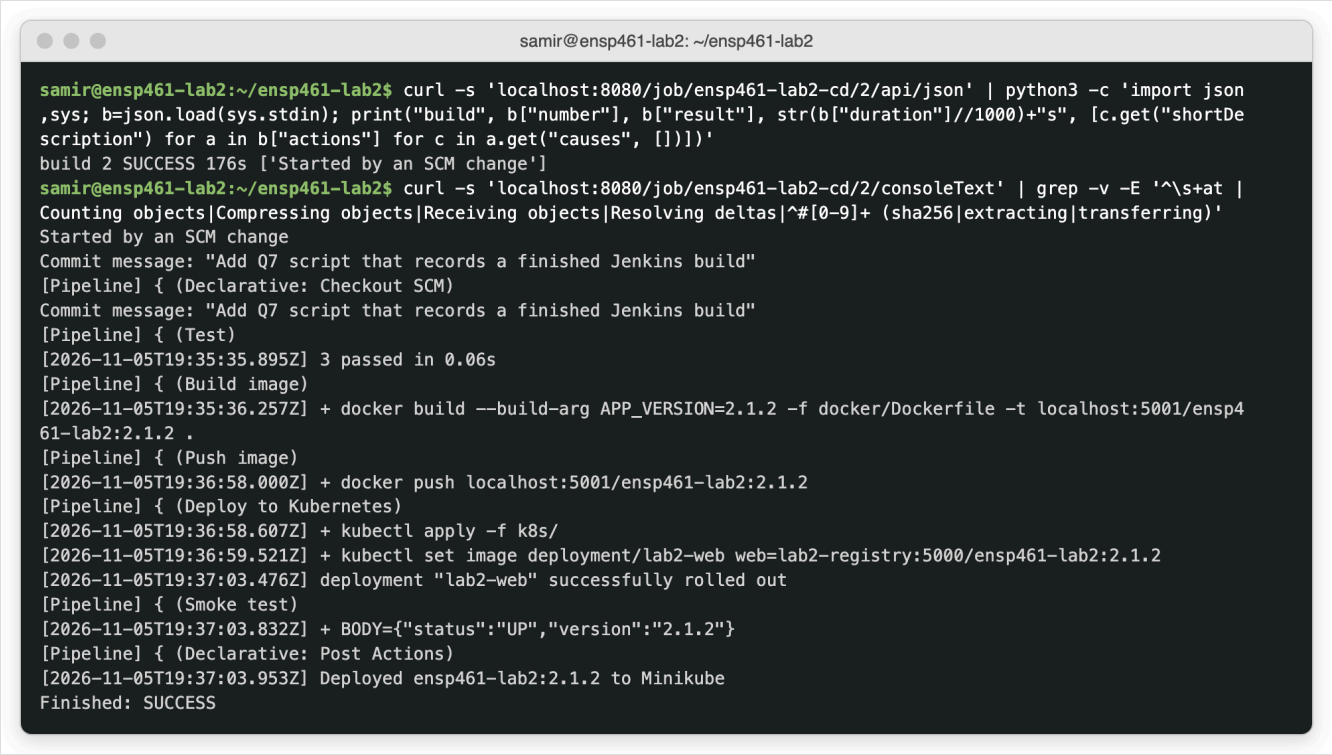


Fig: Build #2 console: SCM trigger, tests, build, push, deploy and smoke test

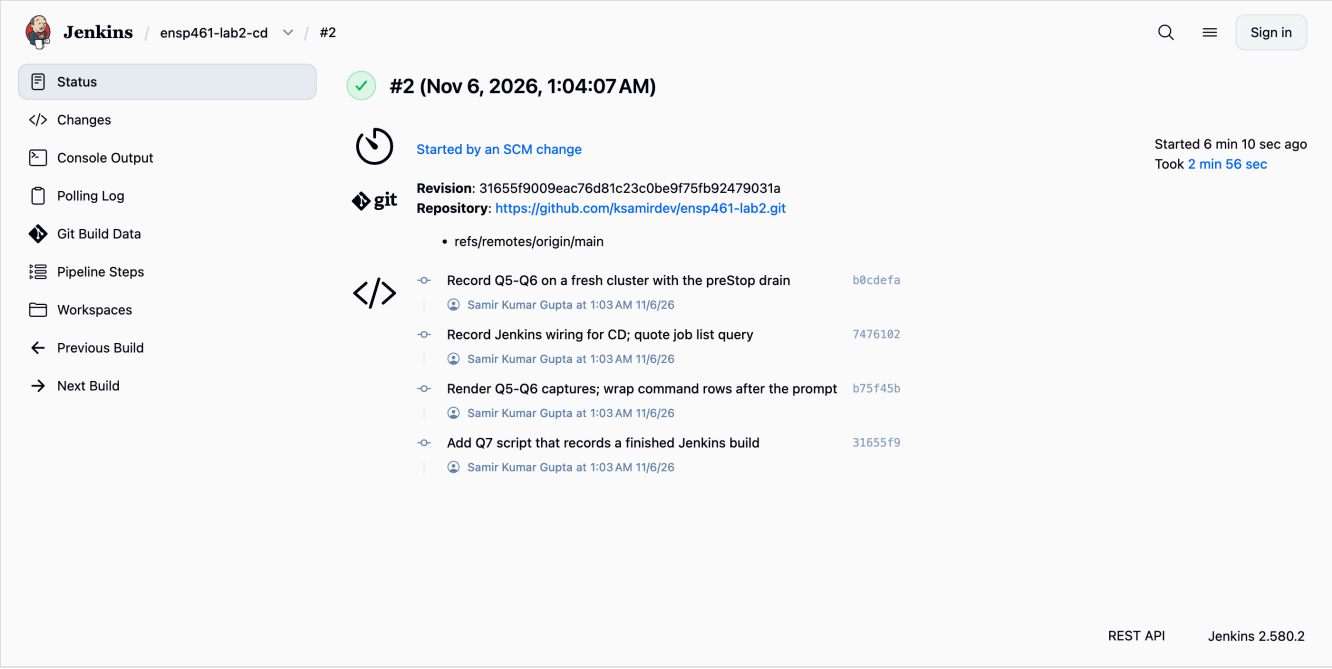


Fig: Build #2 summary: started by an SCM change, commits by Samir Kumar Gupta

Code change deployed automatically

Commit `Report process uptime in /health` added `uptime_seconds` to `/health` with a test. After `git push`, the next poll found the change and build #3 deployed it with no manual step:

```
build 3 SUCCESS 27s ['Started by an SCM change']
$ kubectl get deployment lab2-web -o wide
lab2-web   3/3   3   3   12m   web   lab2-registry:5000/ensp461-lab2:2.1.3   app=lab2-web
$ curl -s http://$(minikube ip):30080/health
{"status":"UP","uptime_seconds":5,"version":"2.1.3"}
$ curl -s localhost:5001/v2/ensp461-lab2/tags/list
{"name":"ensp461-lab2","tags":["2.0.0","latest","2.0.1","2.1.2","2.1.3"]}
```

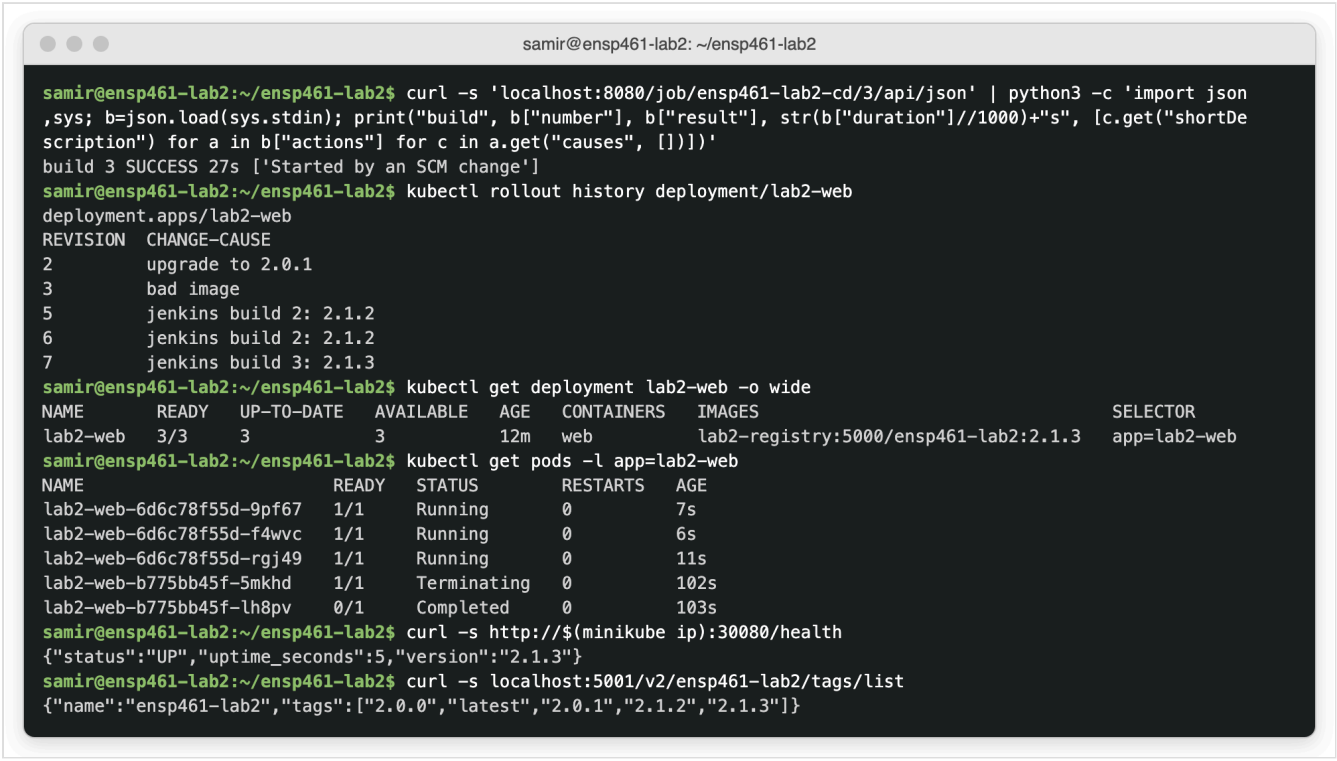


Fig: Build #3: new version live on the cluster, image stored in the registry

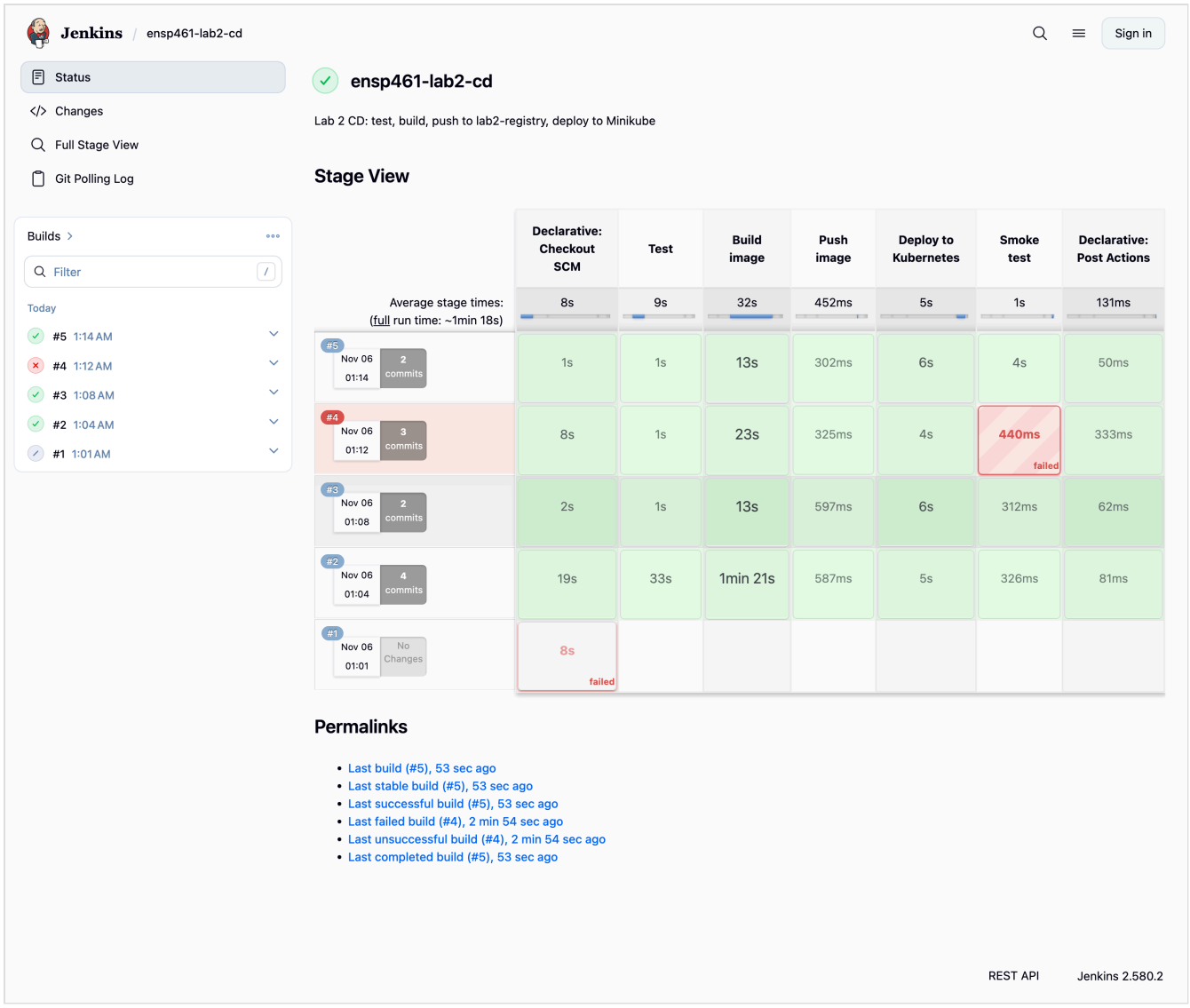


Fig: Jenkins stage view for all builds

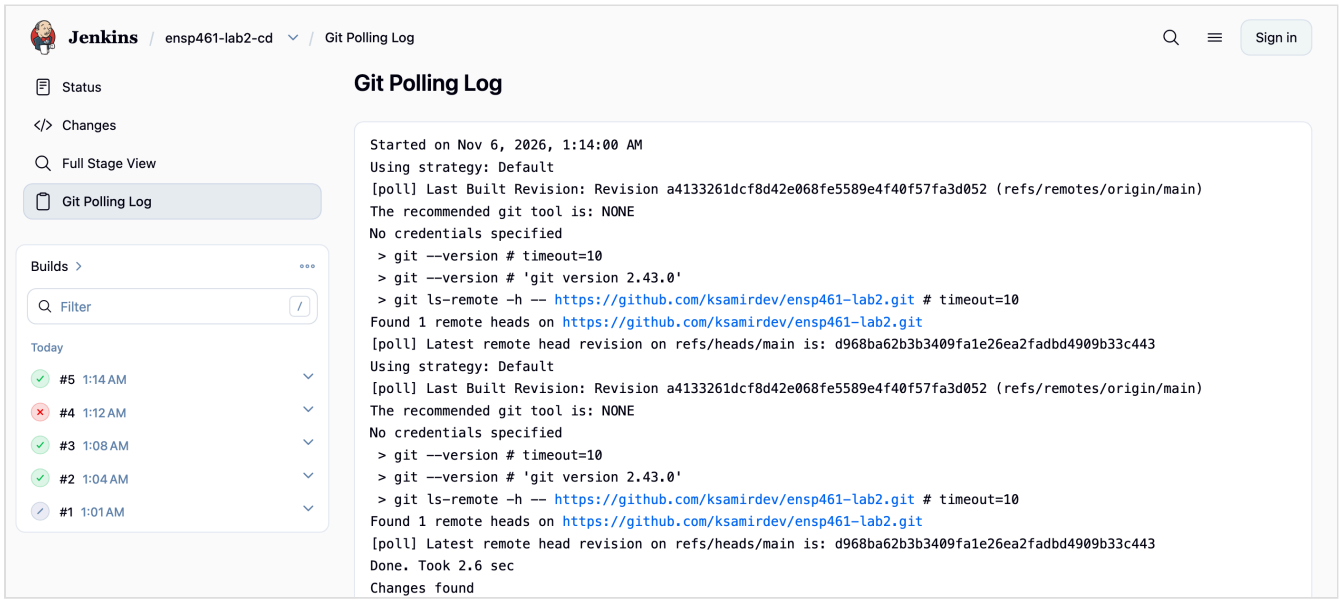


Fig: Git polling log

Problem found: every deploy rolled back first

The first version of the deploy stage ran `kubectl apply -f k8s/` and then `kubectl set image`. After builds #2 and #3 the rollout history showed two revisions for build #2 (5 and 6), and during build #3 an old pod was seen `Completed`:

```
REVISION  CHANGE-CAUSE
5         jenkins build 2: 2.1.2
6         jenkins build 2: 2.1.2
7         jenkins build 3: 2.1.3
```

`kubectl apply` set the image back to `2.0.0` from the manifest, which started a rollout to the old version, and `set image` then started a second rollout to the new one. The stage now pipes the manifest through `sed` to put the build's image in it and applies it once, so each build creates exactly one revision.

Build #4 confirmed the fix: it added one revision (8) instead of two.

Problem found: smoke test raced the drain, and rollback worked

Build #4 itself failed at the smoke test:

```
[19:42:59.207Z] + kubectl rollout status deployment/lab2-web --timeout=180s
[19:43:02.637Z] deployment "lab2-web" successfully rolled out
[19:43:02.970Z] + BODY={"status":"UP","uptime_seconds":263,"version":"2.1.3"}
[19:43:03.187Z] Deployment failed, rolling back to the previous revision
[19:43:03.456Z] + kubectl rollout undo deployment/lab2-web
Finished: FAILURE
```

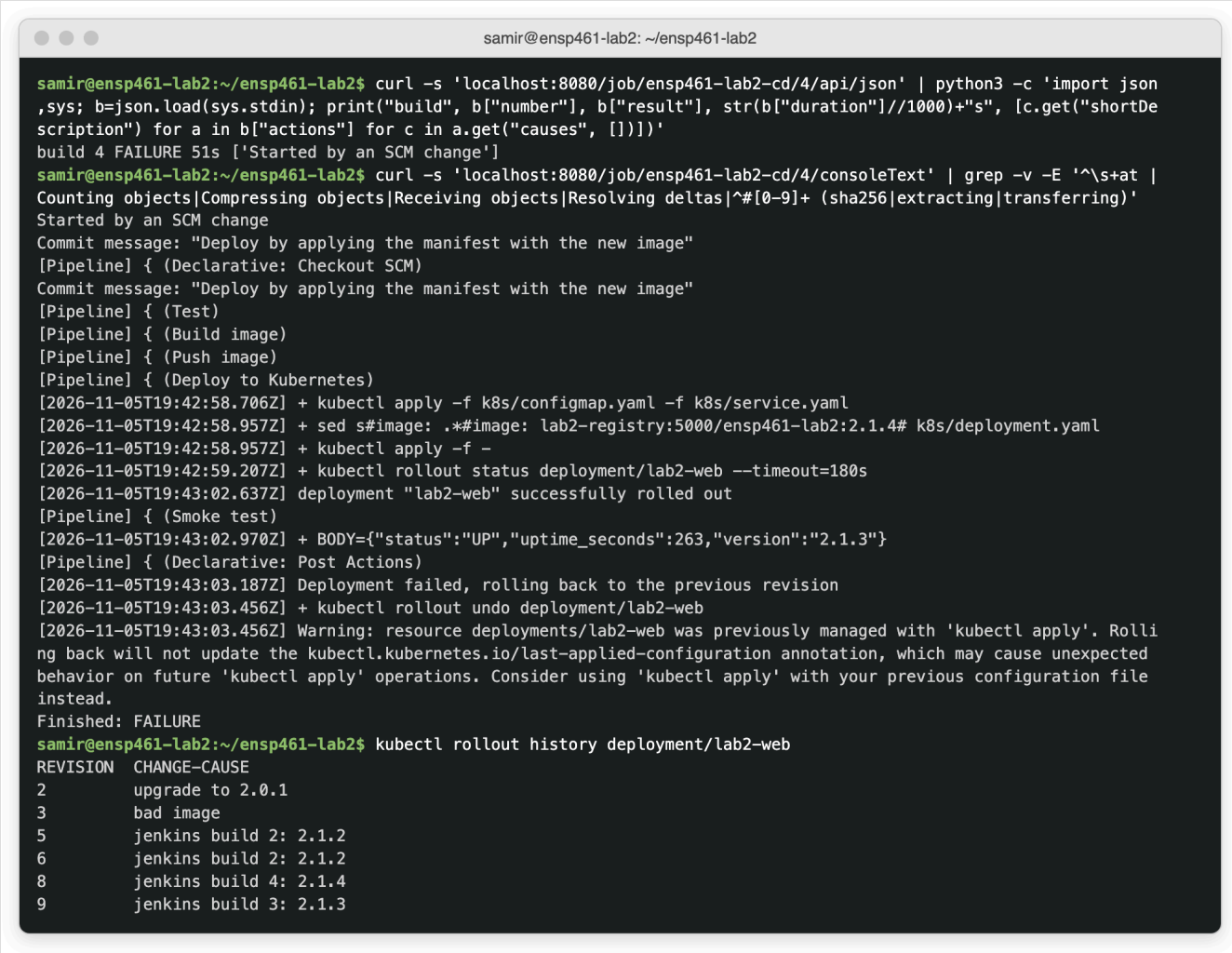


Fig: Build #4: smoke test sees the old version, post step rolls back

`rollout status` reports success once the new pods are ready, but the old pods are still terminating. Because of the `preStop` drain from Q6 they keep answering for 5 s, and the first health check reached one of them (`uptime_seconds: 263`, version 2.1.3). The failure path worked as designed: the `post { failure }` block ran `kubectl rollout undo`, and the cluster went back to 2.1.3 (revision 9) with no manual action.

The smoke test now polls for up to 30 s and requires five consecutive answers with the new version. Build #5 then passed in 30 s:

```
build 5 SUCCESS 30s ['Started by an SCM change']
[19:44:38.414Z] {"status":"UP","uptime_seconds":5,"version":"2.1.5"}
...                                                  (5 consecutive 2.1.5 answers)
$ kubectl rollout history deployment/lab2-web
8         jenkins build 4: 2.1.4
9         jenkins build 3: 2.1.3
10        jenkins build 5: 2.1.5
$ curl -s http://$(minikube ip):30080/health
{"status":"UP","uptime_seconds":6,"version":"2.1.5"}
```

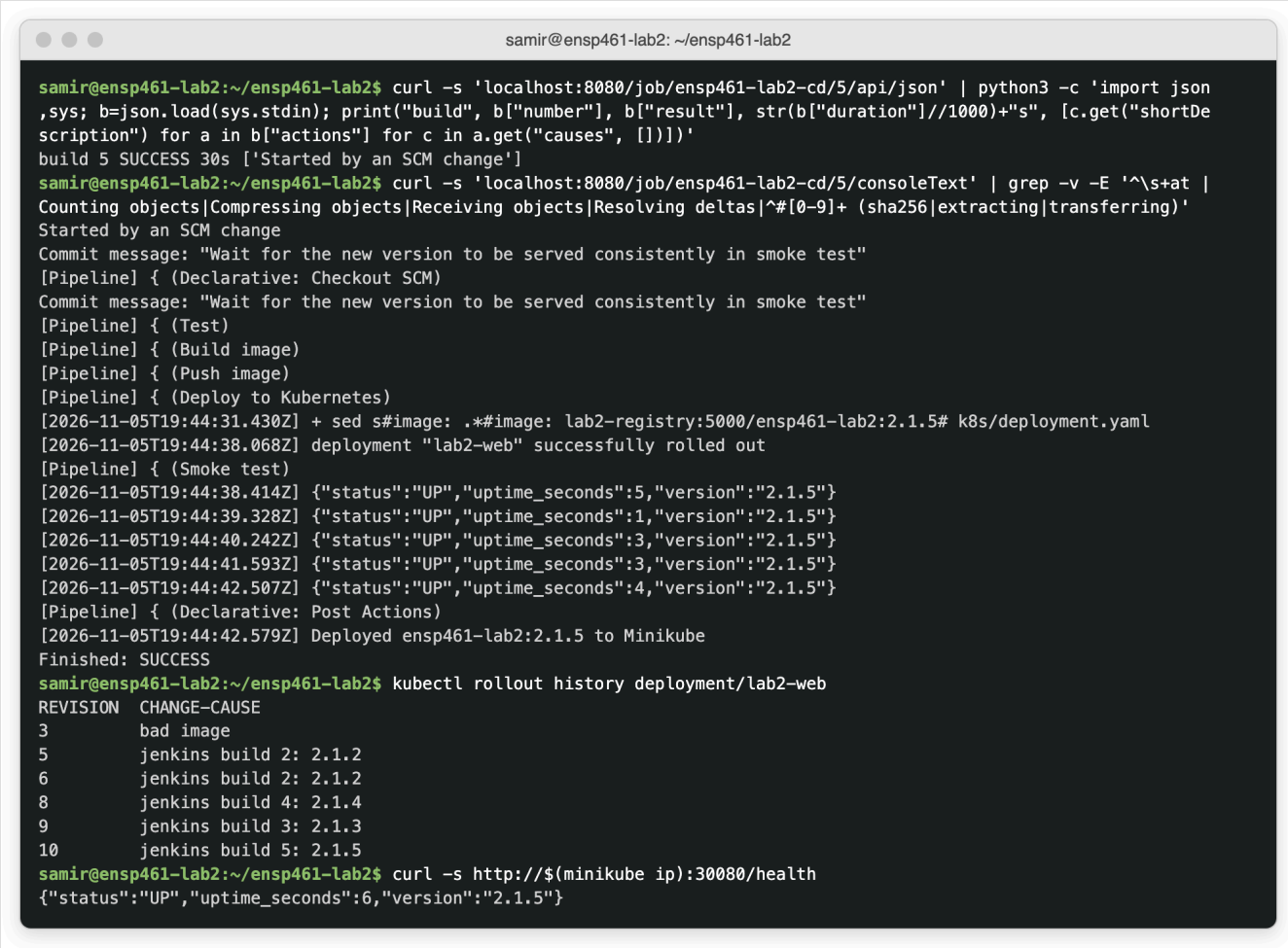


Fig: Build #5: consistent new version, one revision

Result

Every push to `main` is tested, packaged, deployed and verified on the cluster without manual steps, one revision per build. A deploy that does not serve its version is rolled back automatically.